LAPORAN PRAKTIKUM MACHINE LEARNING
K NEAREST NEIGHBORS



Oleh:

Nama : Elsadanika Maura Sitorus

NIM : 211344013

Tanggal Praktikum : 27 Oktober 2023

Tanggal Pengumpulan Laporan : 31 Oktober 2023

PRODI D4 TEKNIK TELEKOMUNIKASI - TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

OKTOBER 2023

1. K Nearest Neighbors

   a) Kode Program dan Fungsinya

Kode Program	Fungsi

`import numpy as np`
`import matplotlib.pyplot as plt`
`import pandas as pd`
`import numpy as np`
`from sklearn import preprocessing`
`%matplotlib inline`
	Baris 6 : perintah khusus unntuk membuat grafik dengan Matplotlib yang akan otomatis ditampilkan pada notebook Jupyter tanpa perlu memanggil 'plt.show()'.

`!wget "https://cf-courses-data.s3.us.cloud-object-storage.a`
`--2023-10-27 07:46:50-- https://cf-courses-data.s3.us.clou`
`Resolving cf-courses-data.s3.us.cloud-object-storage.appdom`
`Connecting to cf-courses-data.s3.us.cloud-object-storage.ap`
`HTTP request sent, awaiting response... 200 OK`
`Length: 36047 (35K) [text/csv]`
`Saving to: 'teleCust1000t.csv'`
`teleCust1000t.csv 100%[==========>] 35.20K --.`
`2023-10-27 07:46:50 (390 KB/s) - 'teleCust1000t.csv' saved`
	Perintah !wget untuk mengunduh file dengan link yang diberikan oleh pemrogram. Output menunjukkan bahwa file teleCust1000t.cvs berhasil diunduh dan disimpan.

`df = pd.read_csv('teleCust1000t.csv')` `df.head()`	Baris 1 : membaca file CSV lalu disimpan dalam bentuk DataFrame dalam variabel df. Baris 2 : menampilkan 5 baris pertama dari DataFrame
`df['custcat'].value_counts()`	df['custcat'] untuk mengakses ke kolom "custcat" dalam df. lalu method value_counts() untuk menghitung jumlah kemunculan setiap nilai pada kolom "custcat"
`df.hist(column='income', bins=50)`	parameter 'bins' menentukan jumlah batang (bin) yang akan digunakan dalam histogram. bins = 50 berarti data dibagi menjadi 50 interval berbeda.
`df.columns`	Perintah untuk mendapat daftar atribut dalam df.

`X = df[['region', 'tenure','age', 'marital', 'address', 'income', 'ed', 'employ','retire', 'gender', 'reside']] .values`
`X[0:5]`
Atribut-atribut tersebut dipilih dan disimpan pada matriks X lalu perintah .values untuk mengambil nilai dari kolom yang dipilih dan dikonversikan menjadi tipe data numpy array.	

`from sklearn.model_selection import train_test_split`
`X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=4)`
`print ('Train set:', X_train.shape, y_train.shape)`
`print ('Test set:', X_test.shape, y_test.shape)`	

Baris 2 : digunakan untuk memisahkan data. Parameter x pada train_test_split adalah atribut yang telah menjadi matrix, y adalah target data (label), test_size adalah proporsi data sebagai set penguji yaitu 20%, random_state adalah pengontrol inisialisasi generator angka acak.

```
X_train_norm = preprocessing.StandardScaler().fit(X_train).transform(X_train.astype(float))
X_train_norm[0:5]
```

StancardScaler adalah method untuk membuat objek, salah satu teknik penskalaan dalam preprocessing untuk memastikan bahwa semua atribut memiliki rata-rata nol dan deviasi standar satu.

```
k = 4
#Train Model and Predict
neigh = KNeighborsClassifier(n_neighbors = k).fit(X_train_norm,y_train)
neigh
```

Baris 1 : program akan mempertimbangkan 4 tetangga terdekat untuk memprediksi kelas data uji.
Baris 2 : membuat objek neigh yang merupakan model K-NN dengan n_neighbors (jumlah tetangga) yang diatur menjadi 4. Kemudian, dilatih (fit) dengan data pelatihan yang telah dinormalisasi (X_train_norm) dan label pelatihan (y_train).

```
yhat = neigh.predict(X_test_norm)
yhat[0:5]
```

Baris 1 : menggunakan model K-NN yang telah dilatih (neigh) untuk melakukan prediksi kelas pada data pengujian yang telah dinormalisasi (X_test_norm).

```
from sklearn import metrics
print("Train set Accuracy: ", metrics.accuracy_score(y_train, neigh.predict(X_train_norm)))
print("Test set Accuracy: ", metrics.accuracy_score(y_test, yhat))
```

mencetak nilai akurasi pada training set dan test set melalui pemanggilan method accuracy_score.

```
Ks = 10
mean_acc = np.zeros((Ks-1))
std_acc = np.zeros((Ks-1))

for n in range(1,Ks):

    #Train Model and Predict
    neigh = KNeighborsClassifier(n_neighbors = n).fit(X_train_norm,y_train)
    yhat=neigh.predict(X_test_norm)
    mean_acc[n-1] = metrics.accuracy_score(y_test, yhat)


    std_acc[n-1]=np.std(yhat==y_test)/np.sqrt(yhat.shape[0])

mean_acc
```

Ks adalah variabel untuk jumlah nilai n_neighbors yang akan diuji dalam

rentang 1-9.
baris 2 dan 3 : pembuatan array numpy yang berisi 0 untuk menyimpan hasil akurasi rata-rata dan deviasi standar akurasi tiap nilai n_neighbors.
baris 9 : deviasi standar akurasi dihitung dengan menggunakan fungsi np.std dan dibagi dengan akar kuadrat dari jumlah data pengujian. Hasil deviasi standar tersebut disimpan dalam array std_acc.

```
plt.plot(range(1,Ks),mean_acc,'g')
plt.fill_between(range(1,Ks),mean_acc - 1 * std_acc,mean_acc + 1 * std_acc, alpha=0.10)
plt.fill_between(range(1,Ks),mean_acc - 3 * std_acc,mean_acc + 3 * std_acc, alpha=0.10,color="green")
plt.legend(('Accuracy ', '+/- 1xstd','+/- 3xstd'))
plt.ylabel('Accuracy ')
plt.xlabel('Number of Neighbors (K)')
plt.tight_layout()
plt.show()
```

Baris 1 : untuk membuat plot garis yang menampilkan akurasi model pada sumbu y dan jumlah tetangga (K) pada sumbu x. Garis ini ditampilkan dalam warna hijau ('g').
Baris 2 : untuk mengisi area antara akurasi rata-rata minus satu deviasi standar dan akurasi rata-rata plus satu deviasi standar. Ini membentuk interval kepercayaan 68% (± 1 deviasi standar) dan diarsir dengan warna hijau muda. Interval kepercayaan ini menunjukkan sejauh mana akurasi model bisa bervariasi.
Baris 4 : menambahkan legenda pada plot, yang menjelaskan makna warna dan garis dalam plot. Legenda ini akan menampilkan teks "Accuracy" untuk garis akurasi, "+/- 1xstd" untuk interval kepercayaan 68%, dan "+/- 3xstd" untuk interval kepercayaan 99.7%.

```
print( "The best accuracy was with", mean_acc.max(), "with k=", mean_acc.argmax()+1)
```

mean_acc.max() mengambil nilai maksimum dari array.
mean_acc.argmax()+1 mengambil indeks dari nilai akurasi tertinggi dalam array mean_acc. Karena indeks dimulai dari 0, maka ditambahkan 1 untuk mendapat nilai K yang sesuai.

b) Input dan Output Program

Input	Output

```
df = pd.read_csv('teleCust1000t.csv')
df.head()
```

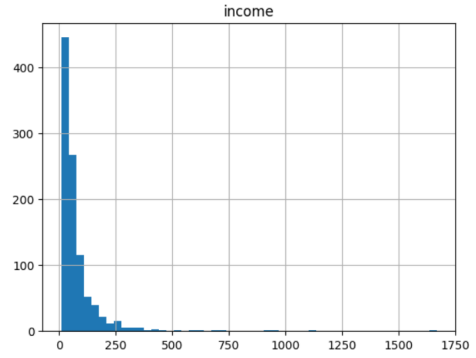
	region	tenure	age	marital	address	income	ed	employ	retire	gender	reside	custcat
0	2	13	44	1	9	64.0	4	5	0.0	0	2	1
1	3	11	33	1	7	136.0	5	5	0.0	0	6	4
2	3	68	52	1	24	116.0	1	29	0.0	1	2	3
3	2	33	33	0	12	33.0	2	0	0.0	1	1	1
4	2	23	30	1	9	30.0	1	2	0.0	0	4	3

```
df['custcat'].value_counts()
```

```
3    281
1    266
4    236
2    217
Name: custcat, dtype: int64
```

```
df.hist(column='income', bins=50)
```

```
array([[<Axes: title={'center': 'income'}>]], dtype=object)
```




```
df.columns
```

```
Index(['region', 'tenure', 'age', 'marital', 'address', 'income', 'ed',
       'employ', 'retire', 'gender', 'reside', 'custcat'],
      dtype='object')
```

```
X = df[['region', 'tenure','age', 'marital', 'address', 'income', 'ed', 'employ','retire', 'gender', 'reside']] .values  #.astype(float)
X[0:5]
```

```
array([[  2.,  13.,  44.,   1.,   9.,  64.,   4.,   5.,   0.,   0.,   2.],
       [  3.,  11.,  33.,   1.,   7., 136.,   5.,   5.,   0.,   0.,   6.],
       [  3.,  68.,  52.,   1.,  24., 116.,   1.,  29.,   0.,   1.,   2.],
       [  2.,  33.,  33.,   0.,  12.,  33.,   2.,   0.,   0.,   1.,   1.],
       [  2.,  23.,  30.,   1.,   9.,  30.,   1.,   2.,   0.,   0.,   4.]])
```

```
y = df['custcat'].values
y[0:5]
```

```
array([1, 4, 3, 1, 3])
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=4)
print ('Train set:', X_train.shape,  y_train.shape)
print ('Test set:', X_test.shape,  y_test.shape)
```

```
Train set: (800, 11) (800,)
Test set: (200, 11) (200,)
```

```
X_train_norm = preprocessing.StandardScaler().fit(X_train).transform(X_train.astype(float))
X_train_norm[0:5]
```

```
array([[-1.28618818e+00, -1.53085556e+00, -8.49354628e-01,
         1.00752834e+00, -7.46393214e-01, -4.83608776e-01,
         1.12305195e+00, -7.02283455e-01, -2.23313158e-01,
        -1.01005050e+00,  1.83304333e+00],
       [-4.79805457e-02,  2.51253999e-01,  3.44957012e-01,
         1.00752834e+00,  2.45326718e-01, -2.71543263e-01,
        -5.40728715e-01, -1.25385370e-04, -2.23313158e-01,
        -1.01005050e+00,  1.83304333e+00],
       [-1.28618818e+00, -7.80493639e-01, -6.90113076e-01,
         1.00752834e+00, -5.21892614e-02, -4.92444839e-01,
        -1.37261905e+00, -8.02591750e-01, -2.23313158e-01,
        -1.01005050e+00,  1.83304333e+00],
       [-4.79805457e-02, -1.53085556e+00, -3.71629972e-01,
        -9.92527915e-01,  1.46154725e-01, -4.74772713e-01,
        -1.37261905e+00, -8.02591750e-01, -2.23313158e-01,
         9.90049504e-01, -9.33814526e-01],
       [ 1.19022709e+00, -2.17722200e-01, -6.10492300e-01,
        -9.92527915e-01, -2.50533248e-01,  2.40948394e-01,
         1.12305195e+00, -4.01358568e-01, -2.23313158e-01,
         9.90049504e-01,  1.14132887e+00]])
```

```
k = 4
#Train Model and Predict
neigh = KNeighborsClassifier(n_neighbors = k).fit(X_train_norm,y_train)
neigh
```

```
KNeighborsClassifier(n_neighbors=4)
```

```
X_test_norm = preprocessing.StandardScaler().fit(X_test).transform(X_test.astype(float))
X_test_norm[0:5]
```

```
array([[-1.13100942, -0.93533648, -0.93817795,  1.02020406, -0.36233308,
         0.17102716, -0.58336825,  0.00666863, -0.21707238, -1.13967126,
         1.97590703],
       [-1.13100942, -1.44823286, -0.13972863, -0.98019606,  0.03474427,
        -0.50017376, -0.58336825, -0.56492821, -0.21707238, -1.13967126,
        -0.90337462],
       [-1.13100942, -0.51569399,  1.53701494,  1.02020406, -0.56087175,
         0.45499678,  0.9619781 , -0.75546048, -0.21707238,  0.87744601,
         1.25608662],
       [-1.13100942, -0.74882871, -0.85833302,  1.02020406, -1.15648777,
        -0.70669712,  0.9619781 , -0.85072662, -0.21707238, -1.13967126,
         0.53626621],
       [-1.13100942,  0.51009876, -1.01802288,  1.02020406, -0.75941043,
         0.17102716,  1.73465128, -0.75546048, -0.21707238, -1.13967126,
         1.25608662]])
```

```
yhat = neigh.predict(X_test_norm)
yhat[0:5]
```

```
array([3, 1, 3, 2, 4])
```

```
from sklearn import metrics
print("Train set Accuracy: ", metrics.accuracy_score(y_train, neigh.predict(X_train_norm)))
print("Test set Accuracy: ", metrics.accuracy_score(y_test, yhat))
```

```
Train set Accuracy:  0.5475
Test set Accuracy:  0.33
```

```
k = 6
neigh6 = KNeighborsClassifier(n_neighbors=k).fit(X_train_norm, y_train)
yhat6 = neigh6.predict(X_test_norm)
print("Train set Accuracy: ", metrics.accuracy_score(y_train, neigh6.predict(X_train_norm)))
print("Train set Accuracy: ", metrics.accuracy_score(y_test,yhat6))
```

```
Train set Accuracy:  0.5125
Train set Accuracy:  0.335
```

```
Ks = 10
mean_acc = np.zeros((Ks-1))
std_acc = np.zeros((Ks-1))

for n in range(1,Ks):

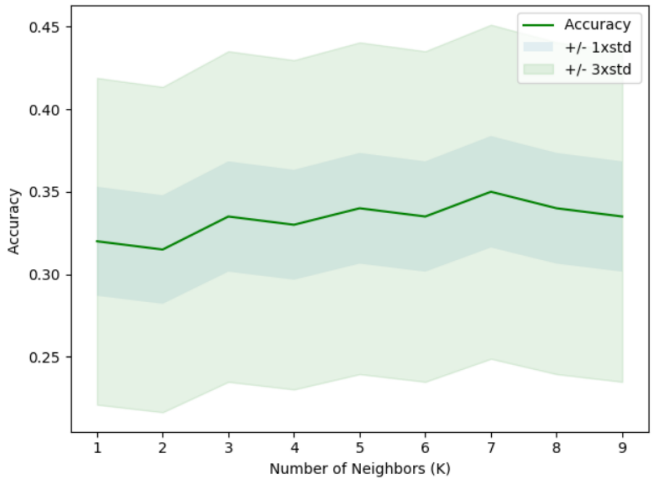
    #Train Model and Predict
    neigh = KNeighborsClassifier(n_neighbors = n).fit(X_train_norm,y_train)
    yhat=neigh.predict(X_test_norm)
    mean_acc[n-1] = metrics.accuracy_score(y_test, yhat)


    std_acc[n-1]=np.std(yhat==y_test)/np.sqrt(yhat.shape[0])

mean_acc
```

```
array([0.32 , 0.315, 0.335, 0.33 , 0.34 , 0.335, 0.35 , 0.34 , 0.335])
```

```
plt.plot(range(1,Ks),mean_acc,'g')
plt.fill_between(range(1,Ks),mean_acc-1*std_acc,mean_acc+1*std_acc,alpha=0.10)
plt.fill_between(range(1,Ks),mean_acc-3*std_acc,mean_acc+3*std_acc,alpha=0.10,color="green")
plt.legend(('Accuracy ', '+/- 1xstd','+/- 3xstd'))
plt.ylabel('Accuracy ')
plt.xlabel('Number of Neighbors (K)')
plt.tight_layout()
plt.show()
```




```
print( "The best accuracy was with", mean_acc.max(), "with k=", mean_acc.argmax()+1)
```

```
The best accuracy was with 0.35 with k= 7
```

c) Analisis

Pada praktikum ini, diberikan program sebagai implementasi K-NN (K-Nearest Neighbors). Secara sederhana, KNN akan mengklasifikasikan suatu data baru tergantung lingkungannya atau data-data lainnya. Data diambil dari file teleCust100t.csv yang merupakan dataset. Data kemudian dibagi menjadi dua bagian yaitu, atribut fitur (x) dan atribut label (y). Data pun dibagi menjadi dua set, yaitu training set (x_train, y_train) dan test set (x_test, y_test) menggunakan metode train_test_split dari library scikit-learn.

Dilakukan proses normalisasi data menggunakan perintah StandardScaler dari sklearn. Normalisasi data adalah langkah penting dalam pra-pemrosesan data yang memastikan bahwa semua atribut memiliki rata-rata nol dan deviasi standar satu. Hal ini membantu dalam menghindari efek atribut dengan skala yang berbeda dalam perhitungan model. Setelah data dinormalisasi, model K-NN dibuat dan dilatih dengan berbagai nilai sebanyak jumlah tetangga (n_neighbors) untuk range 1-9. Walaupun Ks=10, akan tetapi loop dijalankan hingga Ks-1. Hal ini dilakukan dalam loop, setiap model dilatih dengan menggunakan train set yang telah dinormalisasi. Tujuan dari loop ini untuk menemukan nilai n_neighbors dengan akurasi tertinggi.

Setelah didapatkan nilai akurasi, kemudian akurasi tersebut dievaluasi menggunakan metrik akurasi yang mengukur sejauh mana model dapat berhasil memprediksi kelas data dengan benar melalui perintah pemanggilan method accuracy_score dari scikit-learn. Hasil akurasi pada setiap nilai n_neighbors disimpan dalam array mean_acc. Deviasi standar akurasi juga dihitung pada setiap nilai n_neighbors dan disimpan dalam array std_acc.

Proses dilanjutkan dengan visualisasi hasil. Hasil akurasi rata-rata setiap nilai ditampilkan dalam plot garis untuk memahami perubahan akurasi akibat perubahan nilai k. Interval kepercayaan dengan deviasi standar 1 (68%) dan deviasi standar 3 (99.7%) ditampilkan dalam plot memberi gambaran variasi akurasi model. Legenda dan label sumbu ditambahkan ke plot untuk memberikan konteks. Program kemudian mencetak informasi tentang nilai n_neighbors yang menghasilkan akurasi tertinggi pada data pengujian. Hal ini membantu dalam tahap pemilihan nilai n_neighbors yang optimal untuk model K-NN. Pada program ini, terpilih nilai k=7 untuk hasil akurasi terbaik yaitu 0.35. Semakin mendekati 1, artinya akurasi semakin baik.

LAPORAN PRAKTIKUM MACHINE LEARNING
DECISION TREES



Oleh:

Nama : Elsadanika Maura Sitorus

NIM : 211344013

Tanggal Praktikum : 27 Oktober 2023

Tanggal Pengumpulan Laporan : 31 Oktober 2023

PRODI D4 TEKNIK TELEKOMUNIKASI - TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

OKTOBER 2023

2. Decision Tree

a) Kode Program dan Fungsinya

<table>
<tr><th>Kode Program</th><th>Fungsi</th></tr>
<tr><td>

```
my_data = pd.read_csv("drug200.csv", delimiter=",")
my_data[0:5]
```

</td><td>parameter ‘demiliter=”,”’ pada baris pertama menunjukkan bahwa koma (,) digunakan sebagai pemisah antar kolom dalam file csv.</td></tr>
<tr><td colspan="2">

```
X = my_data[['Age', 'Sex', 'BP', 'Cholesterol', 'Na_to_K']].values
X[0:5]
```

variabel x akan berisi fitur yang dipilih dari my_data, perintah .values untuk mengkonversi hasil pemilihan kolom menjadi bentuk array numpy.</td></tr>
<tr><td>

```
from sklearn import preprocessing
le_sex = preprocessing.LabelEncoder()
le_sex.fit(['F','M'])
X[:,1] = le_sex.transform(X[:,1])


le_BP = preprocessing.LabelEncoder()
le_BP.fit([ 'LOW', 'NORMAL', 'HIGH'])
X[:,2] = le_BP.transform(X[:,2])


le_Chol = preprocessing.LabelEncoder()
le_Chol.fit([ 'NORMAL', 'HIGH'])
X[:,3] = le_Chol.transform(X[:,3])

X[0:5]
```

</td><td>- baris 2 : membuat method LabelEncoder() disebut le_sex untuk mengkodekan label pada kolom sex dalam data.
- baris 3 : variabel le_sex mempelajari label unik dalam kolom Sex, yaitu label F (Female) dan M (Male).
- baris 4 : mengambil kolom kedua dalam array X lalu mengganti dengan hasil transformasi dengan objek le_sex. Label F menjadi 0 dan M menjadi 1.</td></tr>
<tr><td>

```
y = my_data["Drug"]
y[0:5]
```

</td><td>Membuat variabel y berisi kolom “Drug” sehingga dalam program ini, label yang ingin diprediksi adalah drug.</td></tr>
<tr><td colspan="2">

```
X_trainset, X_testset, y_trainset, y_testset = train_test_split(X, y, test_size=0.3, random_state=3)
```

random_state = 3 merupakan perintah untuk mengatur seed (bilangan acak awal) untuk memastikan pembagian dataset akan selalu sama jika ingin dijalankan kemudian hari.</td></tr>
<tr><td colspan="2">

```
drugTree = DecisionTreeClassifier(criterion="entropy", max_depth = 4)
drugTree # it shows the default parameters
```

parameter ‘criterion=”entrophy”’ untuk menentukan kinerja untuk membagi node dalam decission tree. Entrophy untuk mengukur ketidakpastian dalam data.
parameter ‘max_depth = 4’ untuk menentukan kedalaman maksimum decision tree untuk mencegah overfitting.</td></tr>
</table>

`print (predTree [0:5])` `print (y_testset [0:5])`	mencetak hasil prediksi dan nilai sebenarnya untuk dibandingkan. Jika hasil prediksi mendekati nilai sebenarnya. maka model bekerja dengan baik.

```
from sklearn import metrics
import matplotlib.pyplot as plt
print("DecisionTrees's Accuracy: ", metrics.accuracy_score(y_testset, predTree))
```

baris 3 : memanggil fungsi accuracy_score dari metrics untuk mengukur akurasi dengan membandingkan parameter y_testset dengan predTree.

`tree.plot_tree(drugTree)` `plt.show()`	baris 1 : untuk membuat plot decision tree dari drugTree. plot_tree merupakan fungsi dari library sklearn.tree

b) Input dan Output Program

Input	Output

```
my_data = pd.read_csv("drug200.csv", delimiter=",")
my_data[0:5]
```

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	F	HIGH	HIGH	25.355	drugY
1	47	M	LOW	HIGH	13.093	drugC
2	47	M	LOW	HIGH	10.114	drugC
3	28	F	NORMAL	HIGH	7.798	drugX
4	61	F	LOW	HIGH	18.043	drugY

```
my_data.shape
```

```
(200, 6)
```

```
X = my_data[['Age','Sex','BP','Cholesterol','Na_to_K']].values
X[0:5]
```

```
array([[23, 'F', 'HIGH', 'HIGH', 25.355],
       [47, 'M', 'LOW', 'HIGH', 13.093],
       [47, 'M', 'LOW', 'HIGH', 10.114],
       [28, 'F', 'NORMAL', 'HIGH', 7.798],
       [61, 'F', 'LOW', 'HIGH', 18.043]], dtype=object)
```

```
from sklearn import preprocessing
le_sex = preprocessing.LabelEncoder()
le_sex.fit(['F','M'])
X[:,1] = le_sex.transform(X[:,1])


le_BP = preprocessing.LabelEncoder()
le_BP.fit([ 'LOW', 'NORMAL', 'HIGH'])
X[:,2] = le_BP.transform(X[:,2])


le_Chol = preprocessing.LabelEncoder()
le_Chol.fit([ 'NORMAL', 'HIGH'])
X[:,3] = le_Chol.transform(X[:,3])

X[0:5]
```

```
array([[23, 0, 0, 0, 25.355],
       [47, 1, 1, 0, 13.093],
       [47, 1, 1, 0, 10.114],
       [28, 0, 2, 0, 7.798],
       [61, 0, 1, 0, 18.043]], dtype=object)
```

```
y = my_data["Drug"]
y[0:5]
```

```
0    drugY
1    drugC
2    drugC
3    drugX
4    drugY
Name: Drug, dtype: object
```

Practice

```
# your code
print("Shape X_trainset:", X_trainset.shape,"&", "Size y_trainset:", y_trainset.shape)
```

```
Shape X_trainset: (140, 5) & Size y_trainset: (140,)
```

```
# your code
print("Shape of X training set {}".format(X_testset.shape),'&','Size of Y training set {}'.format(y_testset.shape))
```

```
Shape of X training set (60, 5) & Size of Y training set (60,)
```

```
drugTree = DecisionTreeClassifier(criterion="entropy", max_depth = 4)
drugTree # it shows the default parameters
```

```
DecisionTreeClassifier(criterion='entropy', max_depth=4)
```

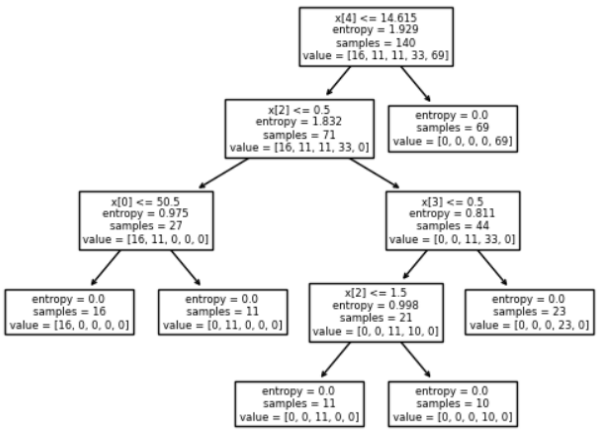
```
drugTree.fit(X_trainset,y_trainset)
```

```
DecisionTreeClassifier(criterion='entropy', max_depth=4)
```

```
print (predTree [0:5])
print (y_testset [0:5])
```

```
['drugY' 'drugX' 'drugX' 'drugX' 'drugX']
40     drugY
51     drugX
139    drugX
197    drugX
170    drugX
Name: Drug, dtype: object
```

```
tree.plot_tree(drugTree)
plt.show()
```



c) Analisis

Pada program ini, digunakan metode decision tree. Terdapat 5 data atribut dari dataset drug200.csv yaitu age, sex, bp (blood pressure), cholesterol, Na_to_k dan 1 kolom kelas yaitu drug. Terdapat 3 kelas klasifikasi obat yaitu drugY, drugC, dan drugX. Dataset ini memiliki dimensi 200 baris dengan 6 kolom yang sudah termasuk dengan label drug. Langkah pertama yang dilakukan adalah menghilangkan kolom label drug karena kolom tersebut merupakan response vector atau target. Atribut age, sex, bp, cholesterol, dan Na_to_k disubstitusikan ke variabel X yang merupakan feature matrix.

Pada dataset, diketahui bahwa atribut sex, bp, dan cholesterol memiliki nilai dalam bentuk kategori-kategori. Sex dikategorikan menjadi female dan male, bp dikategorikan menjadi low, normal, dan high, sedangkan cholesterol dikategorikan sebagai normal dan high. Sklearn Decision Tree tidak bisa memproses variabel kategorial ini, sehingga harus diubah menjadi bentuk numerik melalui fungsi LabelEncoder() yang diambil dari method preprocessing pada library sklearn. Contoh untuk hal ini adalah, mengambil kolom kedua dalam array X (atribut 'Sex') lalu mengganti dengan hasil transformasi dengan objek le_sex. Label F menjadi 0 dan M menjadi 1. Begitupun untuk bp dan cholesterol. Dengan demikian semua data menjadi dalam bentuk numerik.

Data dibagi menjadi 4 parameter yang berbeda melalui train_test_split yaitu X_trainset, X_testset, y_trainset, dan y_testset dengan persentasi data untuk pengujian atau testset sebanyak 30% dan data untuk pelatihan atau train set adalah 70%. Dari total 200 data yang dimiliki, data untuk pelatihan adalah sebanyak 140, sedangkan data untuk pengujian adalah sebanyak 60 data.

Pembentukan decision tree dilakukan menurut pertimbangan purity. Ketika suatu data memiliki purity pembagian label yang jelas tanpa adanya campuran antar label dengan pembagian data tersebut, maka nilai purity nya tinggi. Entropy merupakan nilai ketidakpastian atau uncertainty, semakin tinggi entropy, maka semakin buruk data tersebut terbagi. Sementara information gain ketika semakin besar maka semakin baik. Sehingga hubungan yang

terbentuk adalah semakin kecil entropy maka semakin besar information gain yang dimiliki.

Menurut hasil yang didapatkan dari program, diketahui bahwa accuracy yang dimiliki adalah 0.983 yang sangat mendekati nilai 1, artinya decision tree yang dihasilkan sudah sangat akurat. Pada hasil akhir decision tree, diketahui bahwa semakin dalam decision tree, hasil akurasi yang didapatkan semakin tinggi dengan nilai entropy semakin mendekati 0.